

Real-Time Incident Management & Monitoring System

Project Specification and Collaboration Brief

One-line idea

Build a mini OneUptime / UptimeRobot style platform that monitors services, detects downtime, opens incidents automatically, updates a dashboard in real time, and sends alerts to the right people.

Prepared for	Friend discussion / project planning
Prepared by	Aman Singh
Date	24 June 2026
Project type	Backend-heavy full-stack system design project
Best for	Resume, placement interviews, system design discussion, distributed-systems basics

This document explains what the project solves, the MVP scope, exact features, architecture, database models, APIs, real-time events, worker design, alerting rules, and how two people can build it together without overengineering.

Important scope control

Do not start with Kubernetes, Kafka, microservices, blockchain, or complex distributed tracing. First build a clean monolith plus worker architecture using PostgreSQL, Redis, BullMQ, WebSocket, and Docker Compose.

1. Executive Summary

The project is a real-time incident management and uptime monitoring platform. Users add URLs/services to monitor. A background worker checks those services at fixed intervals. If a service fails repeatedly or becomes slow, the system creates an incident, updates the dashboard live, stores history, and sends alerts through email/webhook/Discord/Slack-like channels.

This is stronger than a normal CRUD project because it includes background jobs, queues, retries, WebSocket real-time updates, incident state machines, time-series style check history, notifications, and production thinking.

Aspect	Specification
Core problem	Teams need to know quickly when their app/API is down or slow.
Main users	Developers, small teams, campus clubs, project maintainers, hackathon teams.
Core flow	Create monitor -> schedule checks -> detect failure -> open incident -> alert -> update status page -> resolve incident.
Realtime depth	Live dashboard updates through WebSocket when status, response time, or incident state changes.
Backend depth	Cron scheduler, Redis queue, BullMQ workers, retry rules, database writes, notification service.
Interview value	Shows knowledge of async processing, resilience, monitoring, queues, WebSocket, DB schema, and system design.

2. What Problem Does It Solve?

When an app is deployed, many things can fail: backend server goes down, database becomes slow, API returns 500 errors, deployment breaks a route, or free-tier hosting sleeps. Without monitoring, the developer only knows when a user complains.

This project solves that by continuously checking important URLs and APIs and converting failures into actionable incidents.

Problem	How this project solves it
Service goes down	Periodic health checks detect failed responses, timeout, or DNS/network error.
Slow API	Stores response time and can mark a monitor degraded if latency crosses a threshold.
Repeated failures	Opens one incident after N consecutive failures instead of spamming incidents.
Team needs live visibility	Dashboard updates immediately through WebSocket events.
Users need transparency	Public status page shows uptime, current incidents, and recent history.
Developer needs history	Check logs and incident timeline help debug when and how failure started.

3. MVP Feature List

The MVP should be small enough to build but deep enough to explain in interviews. The first version should focus on monitoring, incidents, real-time dashboard, and alerting.

Priority	Feature	Specification
P0	Authentication	Google/email auth or simple JWT auth. Users should only see their own monitors and incidents.
P0	Create monitor	User can add name, URL, method, interval, timeout, expected status code, and alert channels.

Priority	Feature	Specification
P0	Worker checks	Background worker performs HTTP checks using Redis/BullMQ jobs and stores results.
P0	Status detection	Monitor states: UP, DOWN, DEGRADED, PAUSED.
P0	Incident automation	Open incident after repeated failures. Auto-resolve after repeated successes.
P0	Real-time dashboard	Socket events update cards, response time, and incident timeline instantly.
P0	Check history	Store status code, latency, error message, timestamp for each check.
P1	Alerts	Email and webhook alerts first; Discord/Slack-like webhook as optional.
P1	Public status page	Shareable page showing current status and active/resolved incidents.
P1	Response time graph	Show response time trend from recent checks.
P2	Team members	Invite members to a project and assign incident ownership.
P2	Maintenance window	Pause alerts during planned maintenance.

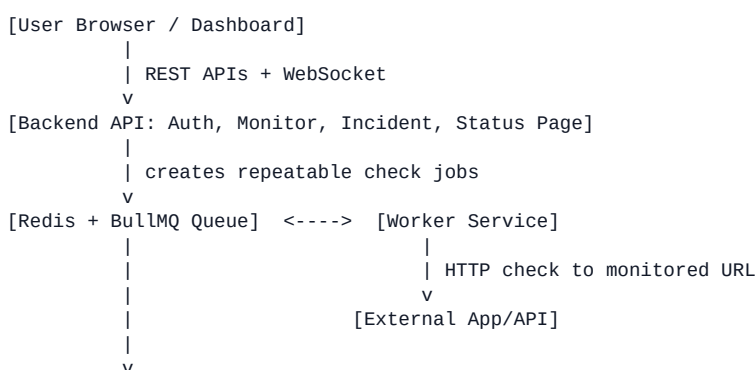
4. Recommended Tech Stack

Use a practical stack that gives real depth without delaying the project. The architecture should be one main API plus separate worker service, not microservices from day one.

Layer	Recommended stack	Reason
Frontend	Next.js + TypeScript + Tailwind	Fast UI, dashboard pages, charts, protected routes.
Backend API	NestJS or Express.js	Clean controllers/services, auth, REST APIs, WebSocket gateway.
Database	PostgreSQL + Prisma	Relational schema fits users, monitors, incidents, checks, alerts.
Queue	Redis + BullMQ	Schedules and processes checks reliably outside the request-response cycle.
Realtime	Socket.IO / WebSocket	Live monitor status, incident updates, and dashboard refresh.
Scheduler	BullMQ repeatable jobs or cron service	Runs checks every N minutes without user request.
Notifications	Email + webhook initially	Enough for MVP; Slack/Discord can be webhook-based.
Charts	Recharts / Chart.js	Response time and uptime visualization.
DevOps	Docker Compose	Run API, worker, PostgreSQL, Redis locally with one command.

5. High-Level Architecture

The most important design idea is separation of concerns. The API should not check URLs directly in the user request. The API creates monitors and schedules jobs. Workers perform checks asynchronously and publish updates.



```

[PostgreSQL: users, monitors, checks, incidents, alerts]
|
| publish status/incident events
v
[WebSocket Gateway + Notification Service]

```

Component	Responsibility
Frontend dashboard	Create monitors, view status cards, incident timeline, response-time chart, status page settings.
Backend API	Auth, CRUD, validation, authorization, incident state APIs, status page APIs.
Queue	Stores scheduled check jobs and retry jobs; decouples monitoring from API requests.
Worker	Runs actual HTTP checks, measures latency, detects errors, writes check result.
Incident engine	Decides when to open, update, acknowledge, or resolve incidents.
WebSocket gateway	Pushes live updates to dashboard rooms/projects.
Notification service	Sends email/webhook alerts when incident state changes.

6. Core Domain Model

The database should make incidents and check history easy to query. Keep the schema simple but interview-explainable.

Entity	Important fields	Purpose
User	id, name, email, avatarUrl, createdAt	Owner/member of projects and monitors.
Project	id, name, slug, ownerId	Groups monitors and incidents for a team/app.
ProjectMember	projectId, userId, role	Supports owner/admin/member roles.
Monitor	id, projectId, name, url, method, intervalSec, timeoutMs, expectedStatus, status, isPaused	Defines what should be checked and how often.
CheckResult	id, monitorId, status, statusCode, responseTimeMs, errorMessage, checkedAt	Stores every health check result.
Incident	id, monitorId, title, status, severity, startedAt, resolvedAt, failureCount	Tracks downtime/degraded periods.
IncidentEvent	id, incidentId, type, message, createdAt	Timeline: opened, alert sent, acknowledged, resolved.
AlertChannel	id, projectId, type, target, isEnabled	Email/webhook/Discord/Slack-like channel config.
NotificationLog	id, incidentId, channelId, status, error, sentAt	Avoid duplicate alerts and debug failed notifications.

7. Suggested Database Schema

Below is a simplified Prisma-style schema. This is not final code, but it is specific enough to guide implementation.

```

model User {
  id          String    @id @default(uuid())
  email       String    @unique
  name        String?
  avatarUrl   String?
  projects    ProjectMember[]
  createdAt   DateTime  @default(now())
}

model Project {
  id          String    @id @default(uuid())
  name        String
  slug        String    @unique
  ownerId     String

```

```

members    ProjectMember[]
monitors    Monitor[]
channels    AlertChannel[]
createdAt   DateTime @default(now())
}

model Monitor {
  id          String    @id @default(uuid())
  projectId   String
  name        String
  url         String
  method      String    @default("GET")
  intervalSec Int        @default(300)
  timeoutMs   Int        @default(5000)
  expectedStatus Int     @default(200)
  status      String    @default("UP") // UP, DOWN, DEGRADED, PAUSED
  failureCount Int       @default(0)
  successCount Int       @default(0)
  isPaused    Boolean   @default(false)
  lastCheckedAt DateTime?
  checks      CheckResult[]
  incidents    Incident[]
  createdAt    DateTime @default(now())
}

model CheckResult {
  id          String    @id @default(uuid())
  monitorId   String
  status      String    // SUCCESS, FAILURE, TIMEOUT, DEGRADED
  statusCode  Int?
  responseTimeMs Int?
  errorMessage String?
  checkedAt   DateTime @default(now())

  @@index([monitorId, checkedAt])
}

model Incident {
  id          String    @id @default(uuid())
  monitorId   String
  title       String
  status      String    @default("OPEN") // OPEN, ACKNOWLEDGED, RESOLVED
  severity    String    @default("HIGH") // LOW, MEDIUM, HIGH, CRITICAL
  startedAt   DateTime @default(now())
  resolvedAt  DateTime?
  events      IncidentEvent[]
}

```

8. REST API Specification

The APIs should be clean and easy to test in Postman. Every endpoint should validate project membership.

Method	Endpoint	Purpose
POST	/auth/login	Login or create user session.
GET	/me	Return current user profile.
POST	/projects	Create a project/team workspace.
GET	/projects	List user's projects.
POST	/projects/:projectId/monitors	Create a monitor and schedule repeatable job.
GET	/projects/:projectId/monitors	List monitors with latest status.
PATCH	/monitors/:monitorId	Edit monitor settings or pause/resume.
POST	/monitors/:monitorId/check-now	Manually trigger an immediate check.
GET	/monitors/:monitorId/checks	Return recent check history for graph/table.
GET	/projects/:projectId/incidents	List open/resolved incidents.
PATCH	/incidents/:incidentId/ack	Acknowledge incident.

Method	Endpoint	Purpose
PATCH	/incidents/:incidentId/resolve	Manually resolve incident.
POST	/projects/:projectId/alert-channels	Add email/webhook alert channel.
GET	/status/:projectSlug	Public status page data.

9. WebSocket Event Specification

WebSocket is used so the dashboard feels real-time. Users join project-specific rooms after authentication.

Event	Direction	Payload idea	When it is used
project:join	Client -> Server	{ projectId }	Dashboard opens and joins project room.
monitor:status_changed	Server -> Client	{ monitorId, status, responseTimeMs, checkedAt }	Worker finishes check and status changes.
monitor:check_result	Server -> Client	{ monitorId, result }	New check result for live graph/table.
incident:opened	Server -> Client	{ incidentId, monitorId, severity }	Failure threshold reached.
incident:updated	Server -> Client	{ incidentId, status, event }	Acknowledged or timeline event added.
incident:resolved	Server -> Client	{ incidentId, resolvedAt }	Success threshold reached or manual resolve.
alert:sent	Server -> Client	{ incidentId, channelType, status }	Alert delivery result.

10. Monitoring and Incident Rules

A monitoring system must avoid false alarms. The important part is not just checking URLs, but deciding when a failure becomes an incident.

Rule	Recommended MVP value	Reason
Interval	5 minutes by default; 1 minute only for demo/local testing	Avoid too many checks on free-tier deployments.
Timeout	5000 ms	If response takes too long, mark timeout/failure.
Failure threshold	2 or 3 consecutive failures	Prevents one random network error from opening incident.
Resolve threshold	2 consecutive successes	Prevents immediate flip-flop between UP and DOWN.
Degraded threshold	Response time greater than 1500 ms	Shows slow service without calling it completely down.
Expected status	200 by default; configurable	Some APIs may return 204 or 301 and still be healthy.
Duplicate incident	One open incident per monitor	Avoids creating many incidents for same outage.
Paused monitor	Do not check and do not alert	Useful during maintenance or development.

11. Worker Flow

The worker is the heart of the project. It makes the project backend-heavy and shows real async processing.

1. API creates monitor in PostgreSQL.
2. API schedules a repeatable BullMQ job: check-monitor:{monitorId}.
3. Worker receives job from Redis.
4. Worker performs HTTP request with timeout.
5. Worker measures response time and captures status code/error.
6. Worker writes CheckResult in PostgreSQL.

7. Worker updates Monitor status and counters.
8. Incident engine decides: open, keep open, acknowledge, resolve, or no action.
9. Notification service sends alert only on important state changes.
10. WebSocket gateway emits live events to project dashboard room.

Pseudo logic

```

async function checkMonitorJob(monitorId) {
  monitor = await db.monitor.findUnique(monitorId)
  if (!monitor || monitor.isPaused) return

  result = await httpCheck(monitor.url, monitor.timeoutMs, monitor.expectedStatus)
  await db.checkResult.create({ monitorId, ...result })

  if (result.success && result.responseTimeMs <= degradedThreshold) {
    monitor.successCount += 1
    monitor.failureCount = 0
    if (monitor.successCount >= RESOLVE_THRESHOLD) resolveOpenIncident(monitor)
    setStatus(monitor, "UP")
  } else if (result.success && result.responseTimeMs > degradedThreshold) {
    setStatus(monitor, "DEGRADED")
    maybeOpenIncident(monitor, "Service is slow")
  } else {
    monitor.failureCount += 1
    monitor.successCount = 0
    if (monitor.failureCount >= FAILURE_THRESHOLD) openIncidentIfNeeded(monitor)
    setStatus(monitor, "DOWN")
  }

  emitSocketUpdate(monitor.projectId, monitor)
}

```

12. Dashboard and UI Pages

The UI should look like a real operations dashboard, not a normal college CRUD screen. Use cards, timelines, status badges, charts, and clean empty states.

Page	Main content
Login	Simple auth page with project branding.
Project dashboard	Overall status, active incidents, uptime summary, latest check results, monitor cards.
Monitors page	List/create/edit monitors, pause/resume, manual check now.
Monitor detail	Response-time graph, uptime percentage, recent checks table, current status.
Incidents page	Open/resolved incidents, severity, timeline, acknowledge/resolve buttons.
Incident detail	Root cause notes, event timeline, alert logs, affected monitor.
Alert channels	Email/webhook setup and test alert button.
Public status page	Customer-facing status, uptime history, active incidents.

13. Alerting Specification

Alerts should be triggered by state changes, not by every failed check. This keeps the project realistic.

Trigger	Send alert?	Message idea
Incident opened	Yes	Monitor API is DOWN. Started at 10:32. Last error: timeout.
Incident acknowledged	Optional	Incident acknowledged by Aman.
Incident resolved	Yes	Monitor API recovered. Downtime: 12 minutes.
Every failed check while incident open	No	Store check result only. Do not spam notifications.
Monitor paused	No	Dashboard update only.
Manual test alert	Yes	Test message to verify channel configuration.

14. Collaboration Plan for Two Friends

This project can be built by two people without conflict if responsibilities are separated by modules.

Person	Ownership	Detailed tasks
Friend A	Backend API + DB	Auth, projects, monitors CRUD, incidents APIs, Prisma schema, authorization, REST docs.
Friend B	Worker + realtime + UI	BullMQ worker, scheduler, HTTP checker, WebSocket events, dashboard, charts, status page.
Both	Integration	Manual check flow, incident state rules, alert testing, README, deployment, demo script.

Recommended Git workflow

Use feature branches: feature/auth, feature/monitors, feature/worker, feature/incidents, feature/realtime-dashboard. Merge only after local testing. Keep a clean README with architecture diagram and setup steps.

15. Build Roadmap

Build the project in phases. Do not add advanced tools until the core monitoring flow works end to end.

Phase	Goal	Deliverable
Phase 1	Foundation	Repo setup, Docker Compose, PostgreSQL, Redis, backend, frontend, auth.
Phase 2	Monitor CRUD	Create/list/edit/pause monitors with validation.
Phase 3	Worker checks	BullMQ repeatable jobs, HTTP checker, CheckResult storage.
Phase 4	Incident engine	Failure threshold, open incident, resolve incident, incident timeline.
Phase 5	Realtime dashboard	Socket project rooms, live status updates, charts.
Phase 6	Alerts	Email/webhook alerts, notification logs, test alert button.
Phase 7	Public status page	Shareable status page with uptime and incidents.
Phase 8	Polish	README, architecture diagram, demo video, load test, error handling.

16. What to Avoid Initially

These are good technologies, but not needed in the first version. Adding them early will delay the project and hide the real learning.

Avoid now	Why
Kubernetes	Too much DevOps overhead for MVP. Docker Compose is enough.
Kafka	BullMQ + Redis is enough for scheduled jobs and retries.
Microservices	API + worker already shows separation. Microservices add complexity without need.
Blockchain	No relevance to incident management.
Real SMS alerts	Can add later, but email/webhook is enough for demo.
Complex KYC/payment	Not related to the core problem.
Distributed tracing	Advanced feature after MVP; not required for placement project.

17. Load Testing and Metrics

This project can show realistic numbers if tested honestly. Do not claim fake 'supports 1 million users'. Instead, show measured metrics from your deployment or local machine.

Metric	How to measure	How to present
API latency	k6/autocannon against create/list endpoints	p50, p95, p99 response time.
Worker throughput	Number of monitor checks processed per minute	Example: 500 checks/min locally with 5 workers.
Queue delay	Time from scheduled job to worker start	Shows if Redis/worker is overloaded.
WebSocket fanout	Number of connected dashboards receiving updates	Example: tested with 100 socket clients.
DB growth	CheckResult rows per monitor per day	Use retention/aggregation plan.
Uptime calculation	Successful checks / total checks	Show 24h/7d/30d uptime.

Honest resume wording

Say: 'Tested worker throughput with X monitor checks/min and measured p95 API latency under Y concurrent users.' Do not say huge numbers unless you actually tested them.

18. Important Edge Cases

Edge cases make the project interview-worthy. Be ready to explain these.

Case	Expected handling
URL is invalid	Validate URL format before saving monitor.
DNS fails	Store error message and count as failure.
Timeout	Abort request after timeoutMs and mark as TIMEOUT.
Expected 200 but got 500	Failure. Store status code.
Expected 204 and got 204	Success because expectedStatus is configurable.
One random failed check	Do not open incident until failure threshold is reached.
Worker crashes	BullMQ can retry job; app should not lose monitor definition.
Redis unavailable	API works partially, but scheduling/checking fails. Show error and retry later.
Database unavailable	Worker cannot store result; fail job and retry.
Free-tier backend sleeps	Mention as limitation. For demo use local Docker or hosting that supports workers.

19. Interview Explanation

A strong interview explanation should focus on why the design choices were made, not just the features.

60-second explanation

I built a real-time incident management and monitoring system. Users can create monitors for APIs or websites. A scheduler pushes check jobs into Redis using BullMQ, and separate workers perform HTTP checks asynchronously. Results are stored in PostgreSQL, incidents are opened or resolved based on consecutive failure/success thresholds, and dashboard clients receive live WebSocket updates. I also added alert channels, check history, response-time graphs, and a public status page. This helped me learn queues, workers, realtime systems, incident state machines, and production-style backend design.

Question	Good answer direction
Why use a queue?	URL checks are background work. Queue decouples API requests from monitoring jobs and supports retries.
Why not check URL inside API?	The user request should return fast; checks run repeatedly even when user is offline.
How do you avoid false alerts?	Use consecutive failure threshold and one open incident per monitor.
How does realtime work?	Authenticated users join project rooms; server emits monitor and incident events after worker updates DB.
How do you scale workers?	Run multiple worker processes consuming from the same BullMQ queue.
How do you store history efficiently?	Keep raw recent checks and aggregate old data into hourly/daily summaries later.

20. Resume Value

This project can become a strong resume project if it is deployed, documented, and measured.

Resume bullet	Why it is strong
Built a real-time incident monitoring platform with URL health checks, automated incident creation, and public status pages.	Clearly explains product value.
Implemented asynchronous monitoring workers using Redis and BullMQ with retry logic and scheduled jobs.	Shows backend depth beyond CRUD.
Designed PostgreSQL schema for monitors, check history, incidents, timelines, alert channels, and notification logs.	Shows database design.
Added WebSocket-based live dashboard updates for monitor state, response-time changes, and incident lifecycle events.	Shows realtime systems.
Measured API latency, worker throughput, and queue delay using load-testing tools and documented results.	Shows engineering maturity.

21. Final Recommendation

This is a very good project to build with a friend if both people want a backend-heavy, system-design-friendly project. It has real product value, clear architecture, and many interview talking points.

The best version is not the most complex version. The best version is a clean MVP that works end to end: create monitor -> worker checks -> failure detected -> incident opened -> alert sent -> dashboard updates live -> incident resolved -> public status page reflects recovery.

Decision

Build it if both people agree to keep the first version focused. Finish the MVP first, then add polish like charts, team roles, maintenance windows, and load-testing metrics.

End of project specification.